

KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY (KUET)

Department of Computer Science and Engineering
Khulna-9203, Bangladesh

News Category Classification and Semantic Analysis Using Traditional NLP Methods

Course Title: Natural Language Processing Laboratory

Course Code: CSE 4122

Academic Session: 2024–2025 (4th Year, 1st Term)

Submitted By:

Adnan Hossain Siraz

Roll No.: 2107068

Department of CSE, KUET

Md. Shakibuzzaman

Roll No.: 2107069

Department of CSE, KUET

Supervised By:

Dr. K. M. Azharul Hasan

Professor, Department of CSE

Khulna University of Engineering &
Technology

Mr. Md. Nazirulhasan Shawon

Lecturer, Department of CSE

Khulna University of Engineering &
Technology

Date of Submission: 23 September 2026

1. Introduction

Automated topic categorization of unstructured news text represents an essential foundational pillar in computational linguistics, modern information retrieval, and algorithmic content recommendation engines [1]. As digital media platforms produce massive volumes of journalistic reporting daily, categorizing continuous news streams into discrete semantic buckets—such as *Business*, *Entertainment*, *Politics*, *Sport*, and *Technology*—is critical for efficient retrieval, trending topic detection, and personalized user consumption[cite: 3, 6]. In academic research and industrial engineering alike, the design of text representation pipelines dictates both the upper bound on classification accuracy and the system’s operational viability under stringent latency and hardware constraints[cite: 4, 6].

1.1. Problem Statement & Linguistic Dynamics

While document-level classification for long-form journalistic articles is well-established, severe degradation in classification performance occurs when traditional models encounter *short-form text* (e.g., news headlines, breaking news tickers, social micro-posts, and push notifications) [4]. Long-form news documents typically contain 200 to 800 words, offering dense repetition of thematic keywords and broad vocabulary distributions that yield reliable document frequencies[cite: 3, 6]. Conversely, short-form headlines consist of only 4 to 12 words[cite: 3, 6]. In such constrained inputs, standard bag-of-words (BoW) representations suffer from extreme mathematical sparsity, acute out-of-vocabulary (OOV) rates, and severe vulnerability to polysemous term overlap[cite: 3, 6].

A central linguistic phenomenon observed during practical evaluation is *semantic shift* and cross-domain lexical borrowing[cite: 6]. In authentic news corpora, vocabulary terms are not strictly partitioned among categories[cite: 3, 6]. For example, financial and commercial terminology frequently appears in sports reporting (e.g., player transfers, franchise buyouts, stadium sponsorship debt, club administration), while geopolitical terms routinely dominate economic coverage (e.g., bilateral trade tariffs, ministerial declarations, sovereign borrowing)[cite: 3, 6]. Under dense vector averaging, short texts lacking explicit domain-exclusive nouns frequently experience representation collapse toward the corpus’s dominant lexical distribution[cite: 3, 6].

1.2. Theoretical Representation Paradigms

Text categorization fundamentally depends on transforming variable-length sequences of discrete lexical tokens into vector representations suitable for statistical optimization[cite: 4, 6]. Historically, two competing paradigms have dominated traditional natural language processing[cite: 4, 6]:

1. **High-Dimensional Sparse Frequency Models:** Methods such as Term Frequency–Inverse Document Frequency (TF-IDF) map documents into high-dimensional vector spaces ($V \sim 10^3\text{--}10^5$), where each dimension corresponds to an exact unigram or higher-order n-gram[cite: 4, 6]. While these representations excel at capturing exact domain keywords and phrases without semantic blurring, they inherently suffer from orthogonal basis independence: synonyms (e.g., “*equity*” and “*stock*”) share zero inner-product overlap[cite: 4, 6].
2. **Continuous Distributed Latent Manifolds:** Continuous embedding techniques, pioneered by Word2Vec [3], map words to low-dimensional continuous vector spaces ($\mathbb{R}^d$, where $d \sim 50\text{--}300$) where cosine distance captures semantic similarity[cite: 4, 6]. When pooling word embeddings into a unified document vector, however, standard unweighted averaging causes *centroid dilution*: ubiquitous and polysemous words distort the document centroid, pulling the resulting vector away from decisive thematic markers[cite: 4, 6].

1.3. Core Objectives & Contributions

This research project delivers a rigorous, formula-first comparative investigation of frequency-based sparse representations and dense semantic vector spaces for multi-class news categorization[cite: 3, 6]. Developed strictly from foundational mathematical equations without black-box library wrappers, this work provides[cite: 4, 6]:

- **Traditional NLP Classification:** Implementation and comparison of Multinomial Naive Bayes and One-vs-Rest Logistic Regression using multiple classical text representations[cite: 6].
- **Multiple Text Representations:** Evaluation of Bag-of-Words, TF-IDF unigram/bigram features, custom Skip-Gram Word2Vec document vectors, and pretrained GloVe document vectors[cite: 6].
- **Zero-Leakage Feature Construction:** Vocabulary, document frequency, and inverse document frequency statistics are learned from the training partition and then applied unchanged to the held-out test set[cite: 6].
- **Semantic Analysis:** Use of Word2Vec/GloVe embeddings and cosine similarity to explore relationships between words in a continuous vector space[cite: 6].
- **Seven-Pipeline Benchmark:** Systematic comparison of seven traditional NLP classification pipelines on the 2,225-document, five-category BBC News dataset using a fixed 445-document held-out test set[cite: 6].
- **Error Analysis and Explainability:** Investigation of representative classification errors, including the short sports headline *“brazil lost 7-1 against germany”*, together with linear feature attribution in the interactive interface[cite: 6].
- **Interactive Demonstration:** Development of an offline Streamlit dashboard for live classification, semantic exploration, and benchmark inspection[cite: 6].

2. Background and Theoretical Formulations

This section establishes the mathematical foundations, governing equations, and optimization objectives underpinning the feature extractors, embedding aggregators, and linear classification heads implemented in this work[cite: 4, 6].

2.1. Term Frequency–Inverse Document Frequency (TF-IDF)

Frequency-based text representation models map unstructured token sequences into a vector space $\mathbb{R}^V$, where V represents the pre-defined vocabulary size[cite: 4, 6]. Raw term frequencies are scaled by their inverse document frequency to discount universally ubiquitous words while elevating domain-discriminative terms [2].

Let $f_{t,d}$ denote the raw occurrence count of term t in document d [cite: 3, 6]. The normalized term frequency is defined as[cite: 3, 6]:

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \quad (1)$$

To prevent division-by-zero singularities and dampen logarithmic scale distortion on finite training collections, we implement the smoothed inverse document frequency[cite: 3, 6]:

$$\text{IDF}(t) = \ln \left(\frac{N_{\text{train}} + 1}{\text{DF}_{\text{train}}(t) + 1} \right) + 1 \quad (2)$$

where N_{train} denotes the total number of articles in the training partition, and $\text{DF}_{\text{train}}(t)$ represents the number of training articles containing term t [cite: 3, 6]. The unnormalized feature value is $v_d[j] = \text{TF}(t_j, d) \times \text{IDF}(t_j)$ [cite: 3, 6]. To eliminate document length bias, the vector is normalized under the Euclidean L_2 norm[cite: 3, 6]:

$$x_d = \frac{v_d}{\|v_d\|_2} = \frac{v_d}{\sqrt{\sum_{j=1}^V (v_d[j])^2}} \quad (3)$$

2.2. Word2Vec: Skip-Gram with Negative Sampling (SGNS)

While TF-IDF captures global document-level statistics, it inherently treats vocabulary terms as orthogonal basis vectors, discarding semantic synonymy and context-dependent relationships[cite: 3, 6]. Distributed representations map tokens into a continuous low-dimensional latent manifold $\mathbb{R}^d$ ($d \ll V$) such that words sharing similar distributional contexts occupy proximal coordinates [3].

In the Skip-Gram architecture, given a sequence of training tokens $w_1, w_2, \dots, w_T$, the model maximizes the average log probability of predicting context words within a sliding window of radius c centered at target token w_t [cite: 3, 6]:

$$\mathcal{L}_{\text{SG}} = \frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} \mid w_t) \quad (4)$$

Because computing exact full-vocabulary softmax normalization is computationally intractable ($\mathcal{O}(|V|)$ per gradient step), we utilize Negative Sampling (SGNS)[cite: 4, 6]. SGNS reformulates context prediction into binary logistic regressions differentiating true observed context pairs from K noise samples[cite: 4, 6]:

$$\mathcal{L}_{\text{SGNS}} = \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \left[\log \sigma \left(v'_{w_{t+j}}{}^\top v_{w_t} \right) + \sum_{k=1}^K \mathbb{E}_{w_{n,k} \sim P_n(w)} \left[\log \sigma \left(-v'_{w_{n,k}}{}^\top v_{w_t} \right) \right] \right] \quad (5)$$

where $v_w, v'_w \in \mathbb{R}^d$ represent target and context embeddings, and $\sigma(z) = \frac{1}{1+e^{-z}}$ is the logistic sigmoid activation[cite: 4, 6].

The negative noise samples are drawn from the smoothed unigram distribution $P_n(w) \propto f(w)^{3/4}$ [cite: 4, 6]. This sub-linear scaling dampens ubiquitous stopwords while elevating the sampling probability of rare and domain-specific terms, ensuring sufficient negative gradient signal without destabilizing parameter updates[cite: 4, 6].

2.3. Document Vector Aggregation: Naive vs. Weighted Centroid

Because Word2Vec models produce token-level vectors v_w , an aggregation mechanism is required to map variable-length documents $d = \{w_1, w_2, \dots, w_{|d|}\}$ into a fixed-dimensional document embedding $v_d \in \mathbb{R}^d$ [cite: 3, 6].

2.3.1. Arithmetic Mean Pooling

The standard baseline computes the unweighted centroid across all in-vocabulary tokens[cite: 3, 6]:

$$v_d^{\text{mean}} = \frac{1}{|d \cap \mathcal{V}|} \sum_{w \in d \cap \mathcal{V}} v_w \quad (6)$$

A critical theoretical failure of arithmetic mean pooling is that ubiquitous, generic vocabulary terms exert equal gravitational pull on the centroid as highly specific topical markers, diluting class-discriminative signals[cite: 3, 6].

2.3.2. TF-IDF Weighted Centroid Aggregation

To suppress common semantic drift, we weight each token vector by its precomputed training TF-IDF significance[cite: 3, 6]:

$$v_d^{\text{tfidf}} = \frac{\sum_{w \in d \cap \mathcal{V}} \text{TF-IDF}(w, d) \cdot v_w}{\sum_{w \in d \cap \mathcal{V}} \text{TF-IDF}(w, d)} \quad (7)$$

If all words in document d are out-of-vocabulary ($d \cap \mathcal{V} = \emptyset$), v_d defaults to the zero vector $\mathbf{0}_d$, prompting the linear classifier to default gracefully to empirical class prior distributions[cite: 3, 6].

2.4. Multinomial Logistic Regression Formulation

For multi-class classification across $C = 5$ categories with feature vector $x \in \mathbb{R}^p$, the linear logits are $z = Wx + b$, where $W \in \mathbb{R}^{C \times p}$ and $b \in \mathbb{R}^C$ [cite: 4, 6]. The posterior class probability distribution is given by the Softmax function[cite: 3, 6]:

$$\hat{y}_c = P(y = c \mid x) = \frac{\exp(z_c)}{\sum_{k=1}^C \exp(z_k)} = \frac{\exp(w_c^\top x + b_c)}{\sum_{k=1}^C \exp(w_k^\top x + b_k)} \quad (8)$$

Optimization minimizes the regularized categorical cross-entropy loss over m training instances[cite: 3, 6]:

$$\mathcal{J}(W, b) = -\frac{1}{m} \sum_{i=1}^m \sum_{c=1}^C y_{i,c} \log \hat{y}_{i,c} + \frac{\lambda}{2} \sum_{c=1}^C \|w_c\|_2^2 \quad (9)$$

where $y_{i,c} \in \{0, 1\}$ represents the ground-truth one-hot class label, and λ is the L_2 Ridge regularization penalty[cite: 4, 6]. Evaluating the gradient with respect to parameters yields the closed-form batch update equation[cite: 4, 6]:

$$\nabla_W \mathcal{J} = \frac{1}{m} (\hat{Y} - Y)^\top X + \lambda W, \quad \nabla_b \mathcal{J} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) \quad (10)$$

where $X \in \mathbb{R}^{m \times p}$ is the design matrix, $Y \in \mathbb{R}^{m \times C}$ is the one-hot target matrix, and $\hat{Y} \in \mathbb{R}^{m \times C}$ represents predicted softmax posteriors[cite: 4, 6].

2.5. One-vs-Rest (OvR) Binary Decomposition

In the One-vs-Rest decomposition, C independent binary classifiers are trained, each separating category c ($y^{(c)} = 1$) from all remaining classes ($y^{(c)} = 0$)[cite: 4, 6]. Logits are clipped for numerical stability ($z_c \in [-250, 250]$) with sigmoid activation $\hat{p}_c = \sigma(z_c)$ [cite: 4, 6]. Parameters are optimized via batch gradient descent with Ridge penalty $\nabla_{w_c} \mathcal{J} = \frac{1}{m} X^\top (\hat{p}_c - y^{(c)}) + \lambda w_c$ [cite: 4, 6]. Normalized class probabilities are obtained via $P(y = c \mid x) = \sigma(z_c) / \sum_k \sigma(z_k)$, and linear feature attribution for token j toward category c is given by $\text{Attr}_c(j) = x_d[j] \cdot w_c[j]$ [cite: 4, 6].

3. Dataset, Corpus, and Preprocessing

3.1. Corpus Description

The experimental investigation is conducted on the official BBC News text classification benchmark, comprising 2,225 authentic journalistic articles published between 2004 and 2005 across five thematic desks[cite: 3, 6]. The corpus contains zero missing values and zero duplicate articles[cite: 3, 6].

Table 1: Dataset distribution across categories and experimental partitions.

Category	Total Articles	Proportion (%)	Train Split (80%)	Held-Out Test (20%)
Sport	511	22.97%	409	102
Business	510	22.92%	408	102
Politics	417	18.74%	333	84
Tech	401	18.02%	321	80
Entertainment	386	17.35%	309	77
Total	2,225	100.00%	1,780	445

3.2. Preprocessing Pipeline & Data Leakage Prevention

Text normalization operates through a structured multi-stage pipeline[cite: 3, 6]:

1. **HTML and Escaped Character Stripping:** Regular expression cleaning removes residual markup tags and newline formatting artifacts[cite: 3, 6].
2. **Case Normalization:** All characters are converted to lowercase[cite: 3, 6].
3. **Tokenization Protocols:** We establish two distinct tokenization configurations to evaluate digit sensitivity[cite: 3, 6]:
 - *Configuration A (Standard):* Regex `\b[a-z]{2,}\b`[cite: 3, 6]. Strips all digits and isolated punctuation[cite: 3, 6].
 - *Configuration B (Scoreline Preserving):* Regex `\b[a-z0-9]+(?:-[a-z0-9]+)*\b`[cite: 3, 6]. Retains hyphenated athletic scorelines (e.g., “7-1”), currency expressions, and numerical metrics[cite: 3, 6].
4. **Stopword Removal & Lemmatization:** NLTK standard English stopwords (179 words) are removed[cite: 3, 6]. Remaining tokens are reduced to morphological root lemmas via the WordNet Lemmatizer[cite: 3, 6].
5. **Strict Zero-Leakage Splitting:** Partitioning is executed via stratified random sampling (`random_state = 42`)[cite: 3, 6]. The training set ($N = 1,780$) is utilized exclusively to fit vocabulary $\mathcal{V}$, compute document frequencies $DF_{\text{train}}(t)$, and calculate $IDF_{\text{train}}(t)$ [cite: 3, 6]. The held-out test partition ($N = 445$) is projected strictly using frozen training parameters; previously unseen tokens are mapped to zero entries[cite: 3, 6].

4. Proposed Methodology

The end-to-end classification system incorporates modular dual-path feature extraction paired with pure-NumPy batch gradient descent classifiers[cite: 4, 6].

4.1. Architectural Overview

The system is organized around two complementary NLP tasks: multi-class news classification and semantic analysis[cite: 6]. For classification, several traditional representations are evaluated with Multinomial Naive Bayes and One-vs-Rest Logistic Regression[cite: 6]. For semantic analysis, Word2Vec and GloVe embeddings are explored using cosine similarity and nearest-neighbor inspection[cite: 6].

- **Statistical Classification Path:** Count/Bag-of-Words features are classified using Multinomial Naive Bayes[cite: 6].

- **Sparse Linear Classification Path:** Unigram and unigram+bigram TF-IDF features are classified using from-scratch One-vs-Rest Logistic Regression[cite: 6].
- **Dense Semantic Classification Path:** 100-dimensional custom Skip-Gram Word2Vec embeddings are aggregated by mean pooling or TF-IDF-weighted pooling and classified using One-vs-Rest Logistic Regression[cite: 6].
- **Pretrained Embedding Baseline:** 50-dimensional pretrained GloVe vectors are aggregated by mean or TF-IDF weighting and evaluated with One-vs-Rest Logistic Regression[cite: 6].
- **Semantic Exploration Path:** Word2Vec and GloVe word vectors are compared through cosine similarity and nearest-neighbor analysis[cite: 6].

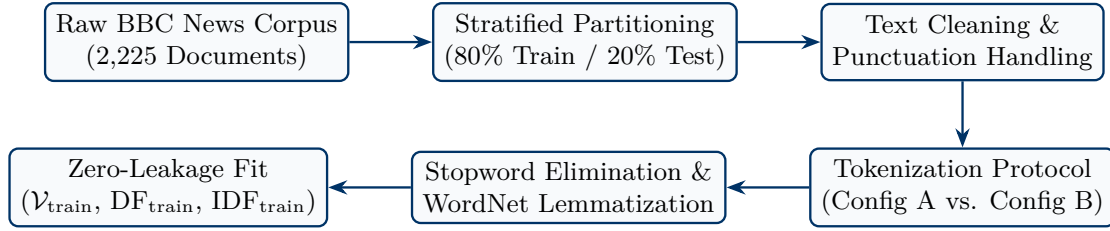


Figure 1: Flowchart illustrating the data preparation, tokenization protocol, and zero-leakage training isolation pipeline.

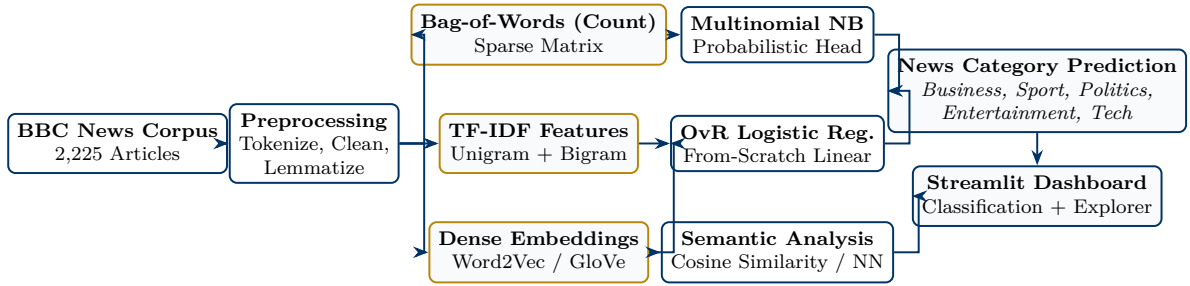


Figure 2: Overall system architecture showing the classification pipelines and the complementary semantic analysis component.

Sparse N-Gram TF-IDF Vector ($x \in \mathbb{R}^{8000}$): Orthogonal, High-Dimensional, Exact Lexical Tokens

0.0	0.42	0.0	0.0	...	0.71	0.0	...	0.38	0.0	0.0
-----	------	-----	-----	-----	------	-----	-----	------	-----	-----

Dense TF-IDF Weighted Centroid ($v_d \in \mathbb{R}^{100}$): Continuous, Distributed Latent Manifold

+0.18	-0.35	+0.04	-0.12	...	+0.41	-0.08	+0.29
-------	-------	-------	-------	-----	-------	-------	-------

Figure 3: Structural comparison between high-dimensional sparse n-gram representation and continuous dense centroid embeddings.

4.2. Formal Procedural Algorithms

To guarantee mathematical reproducibility, the algorithmic procedures for feature extraction and multi-class optimization are formalized below[cite: 4, 6].

Algorithm 1: TF-IDF Weighted Dense Centroid Feature Extraction

```
1: Input: Document token sequence  $d = [w_1, w_2, \dots, w_{|d|}]$ , vocabulary dictionary  $\mathcal{V}$ , embedding matrix  $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$ , precomputed smoothed inverse document frequency array  $\text{IDF} \in \mathbb{R}^{|\mathcal{V}|}$ [cite: 4, 6].
2: Output: Normalized document embedding vector  $v_d \in \mathbb{R}^d$ [cite: 4, 6].
3: Initialize accumulator vector  $S \leftarrow \mathbf{0}_d$  and scalar weight accumulator  $W \leftarrow 0.0$ [cite: 4, 6].
4: Count token frequencies  $f_w \leftarrow \text{CountOccurrences}(w, d)$  for each distinct token  $w \in d$ [cite: 4, 6].
5: for each unique token  $w \in d$  do
6:   if  $w \in \mathcal{V}$  then
7:     Retrieve vocabulary index  $j \leftarrow \mathcal{V}[w]$  and embedding row vector  $e_j \leftarrow \mathbf{E}[j, :]$ [cite: 4, 6].
8:     Compute normalized term frequency  $\text{TF}(w, d) \leftarrow f_w / |d|$ [cite: 4, 6].
9:     Compute composite token significance weight  $\omega_w \leftarrow \text{TF}(w, d) \times \text{IDF}[j]$ [cite: 4, 6].
10:    Accumulate weighted vector:  $S \leftarrow S + \omega_w \cdot e_j$ [cite: 4, 6].
11:    Accumulate total weight:  $W \leftarrow W + \omega_w$ [cite: 4, 6].
12:   end if
13: end for
14: if  $W > 0$  then
15:   Compute centroid:  $v_d \leftarrow S / W$ [cite: 4, 6].
16:   Compute Euclidean norm:  $\|v_d\|_2 \leftarrow \sqrt{\sum_{k=1}^d v_d[k]^2}$ [cite: 4, 6].
17:   if  $\|v_d\|_2 > 0$  then
18:      $v_d \leftarrow v_d / \|v_d\|_2$  // Euclidean  $L_2$  Normalization[cite: 4, 6]
19:   end if
20: else
21:   Set zero vector fallback:  $v_d \leftarrow \mathbf{0}_d$  // Graceful fallback for out-of-vocabulary inputs[cite: 4, 6]
22: end if
23: return  $v_d$ 
```

Algorithm 2: One-vs-Rest (OvR) Logistic Regression via Batch Gradient Descent

```
1: Input: Training design matrix  $X \in \mathbb{R}^{m \times p}$ , one-hot label matrix  $Y \in \mathbb{R}^{m \times C}$ , learning rate  $\eta$ , regularization parameter  $\lambda$ , max epochs  $T_{\max}$ , tolerance  $\epsilon$ [cite: 4, 6].
2: Output: Optimized weight matrix  $W^* \in \mathbb{R}^{C \times p}$  and bias vector  $b^* \in \mathbb{R}^C$ [cite: 4, 6].
3: Initialize weights  $W \leftarrow \mathbf{0}_{C \times p}$  and biases  $b \leftarrow \mathbf{0}_C$ [cite: 4, 6].
4: for each category  $c \in \{1, 2, \dots, C\}$  do
5:   Extract binary target vector  $y^{(c)} \leftarrow Y[:, c] \in \{0, 1\}^m$ [cite: 4, 6].
6:   for epoch  $t \leftarrow 1$  to  $T_{\max}$  do
7:     Compute linear logits:  $z \leftarrow Xw_c + b_c \in \mathbb{R}^m$ [cite: 4, 6].
8:     Clip logits to prevent floating-point overflow:  $z_{\text{clip}} \leftarrow \text{clip}(z, -250.0, 250.0)$ [cite: 4, 6].
9:     Compute sigmoid probabilities:  $\hat{p} \leftarrow 1.0 / (1.0 + \exp(-z_{\text{clip}}))$ [cite: 4, 6].
10:    Compute prediction residual:  $r \leftarrow \hat{p} - y^{(c)} \in \mathbb{R}^m$ [cite: 4, 6].
11:    Compute weight gradient with  $L_2$  Ridge penalty:  $g_w \leftarrow \frac{1}{m} X^\top r + \lambda w_c$ [cite: 4, 6].
12:    Compute bias gradient:  $g_b \leftarrow \frac{1}{m} \sum_{i=1}^m r_i$ [cite: 4, 6].
13:    Update parameters:  $w_c \leftarrow w_c - \eta g_w$ ,  $b_c \leftarrow b_c - \eta g_b$ [cite: 4, 6].
14:    if  $\|g_w\|_2 < \epsilon$  then
15:      break // Convergence threshold satisfied[cite: 4, 6]
16:    end if
17:   end for
18: end for
19: return  $W^* \leftarrow W$ ,  $b^* \leftarrow b$ 
```

5. Training and Experimental Setup

All models were developed, trained, and validated locally under Python 3.13.15 on a Windows 64-bit environment without GPU dependencies[cite: 3, 6]. Table 2 outlines the systematic hyperparameter configurations established across each module[cite: 6].

Table 2: Systematic hyperparameter configuration across all experimental modules.

Subsystem	Hyperparameter	Parameter	Selected Configuration	Value
TF-IDF Extractor	N-Gram Boundaries		Unigram (1, 1) and Unigram + Bigram (1, 2)	
	Vocabulary Dimension (V)		8,000 (Bigram) / 5,000 (Unigram)	
	Minimum Document Frequency		$DF_{\min} = 2$	
	Vector Normalization		Euclidean L_2 Normalization	
Custom Word2Vec	Embedding Dimensionality (d)		100	
	Context Window Radius (c)		5 tokens	
	Negative Sampling Rate (K)		5 noise tokens per positive pair	
	Subsampling Threshold		$t_{\text{sub}} = 10^{-3}$	
	Training Epochs		50	
	Optimizer & Learning Rate		Adam ($\alpha = 0.005$, $\beta_1 = 0.9$, $\beta_2 = 0.999$)	
Logistic Regression (OvR)	Learning Rate (η)		0.5	
	Iteration Limit		1,000 batch iterations	
	Regularization Parameter (λ)		10^{-4} (Ridge L_2 Penalty)	
	Logit Bounds		$[-250, 250]$ (Numerical Clipping)	
	Stopping Tolerance		$\epsilon = 10^{-6}$	
Pretrained Transfer	GloVe Baseline		50-dimensional Wikipedia+Gigaword vector cache	

5.1. Evaluation Protocol

Model generalization is assessed on the held-out test split of 445 samples across[cite: 3, 6]:

- **Accuracy:** Overall ratio of correctly predicted documents to total test instances[cite: 3, 6].
- **Macro-Averaged Precision, Recall, and F1-Score:** Arithmetic mean of per-class metrics, treating each class with equal importance[cite: 3, 6]:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C \frac{2 \cdot P_c \cdot R_c}{P_c + R_c} \quad (11)$$

- **Weighted F1-Score:** F1-scores weighted by class support $\sum_{c=1}^C \frac{N_c}{N} \text{F1}_c$ [cite: 3, 6].

6. Results and Quantitative Evaluation

6.1. Benchmark Performance Matrix

Table 3 summarizes empirical performance across all seven investigated pipeline configurations on the identical 445-sample held-out test set[cite: 3, 6].

Table 3: Benchmark performance comparison across all seven traditional NLP pipelines on the held-out test split ($N = 445$).

Pipeline ID	Model Architecture	Representation / Pooling	Accuracy	Macro P	Macro R	Macro F1	Weighted F1
Pipeline A	Naive Bayes	Count / Bag-of-Words	97.53%	97.47%	97.51%	97.48%	97.53%
Pipeline G	Logistic Regression	TF-IDF (Unigram + Bigram)	97.30%	97.48%	97.18%	97.31%	97.32%
Pipeline F	Logistic Regression	TF-IDF (Unigram)	97.08%	97.23%	97.00%	97.10%	97.09%
Pipeline B	Logistic Regression	Custom Word2Vec (Mean)	96.40%	96.36%	96.26%	96.30%	96.41%
Pipeline C	Logistic Regression	Custom Word2Vec (TF-IDF)	96.18%	96.15%	96.00%	96.05%	96.18%
Pipeline D	Logistic Regression	Pretrained GloVe (Mean)	76.63%	82.04%	74.79%	75.73%	76.06%
Pipeline E	Logistic Regression	Pretrained GloVe (TF-IDF)	68.31%	78.04%	66.17%	67.47%	67.74%

6.2. Cross-Architectural Class-Wise Comparison

To analyze how distinct representation topologies affect specific topic categories, Table 4 compares per-class F1-scores across the three primary high-performing configurations: Pipeline A

(Count Naive Bayes), Pipeline G (TF-IDF Unigram+Bigram Logistic Regression), and Pipeline C (Custom Word2Vec TF-IDF Centroid Logistic Regression)[cite: 4, 6].

Table 4: Per-class F1-score performance comparison across key foundational architectures on the held-out test split ($N = 445$).

Category	Test Support	Pipeline A (Naive Bayes)	Pipeline G (TF-IDF + LR)	Pipeline C (Word2Vec + LR)
Business	102	95.15%	94.74%	93.72%
Entertainment	77	98.04%	97.40%	96.73%
Politics	84	96.97%	97.59%	95.81%
Sport	102	100.00%	100.00%	100.00%
Tech	80	97.50%	96.82%	94.27%
Macro-Averaged F1	445	97.48%	97.31%	96.05%

The comparative breakdown reveals several key linguistic insights[cite: 4, 6]:

- **Impenetrable Athletic Boundary:** All three architectures achieved a flawless 100.00% F1-score on *Sport*[cite: 4, 6]. Journalistic sports reporting possesses an exceptionally distinct, insular lexicon (“*championship*”, “*striker*”, “*olympic*”, “*coach*”, “*goal*”) that generates virtually zero lexical overlap with other news desks in full-length articles[cite: 4, 6].
- **The Business–Tech Ambiguity:** The largest performance degradation in dense vector models occurs in *Business* (93.72%) and *Tech* (94.27%)[cite: 4, 6]. Technology reporting routinely covers quarterly corporate earnings of software conglomerates, patent litigation, and venture capital, causing dense centroid embeddings to suffer from semantic bleed between commercial and technological vector coordinates[cite: 4, 6].
- **Sparse Feature Precision in Politics:** Pipeline G achieved the highest F1-score on *Politics* (97.59%), outperforming both Naive Bayes (96.97%) and Word2Vec (95.81%)[cite: 4, 6]. Bigram phrases such as “*prime_minister*”, “*general_election*”, and “*labour_party*” establish unambiguous linear boundaries that dense averaging partially attenuates[cite: 4, 6].

6.3. Granular Class-Wise Evaluation of Pipeline G

Table 5 presents granular per-category precision, recall, and support metrics for the primary linear classifier operating on Unigram + Bigram TF-IDF representations (Pipeline G)[cite: 3, 6].

Table 5: Detailed class-wise performance breakdown for Pipeline G (TF-IDF Unigram + Bigram $\rightarrow$ Logistic Regression).

Category	Precision	Recall	F1-Score	Test Support	Correctly Classified
Business	92.52%	97.06%	94.74%	102	99
Entertainment	97.40%	97.40%	97.40%	77	75
Politics	98.78%	96.43%	97.59%	84	81
Sport	100.00%	100.00%	100.00%	102	102
Tech	98.70%	95.00%	96.82%	80	76
Overall Macro	97.48%	97.18%	97.31%	445	433

6.4. Confusion Matrix Analysis & Cross-Model Error Distribution

To systematically contrast error patterns between sparse frequency representations and dense latent manifolds, we analyze the 5×5 confusion matrices for both Pipeline G (Table 6) and Pipeline C (Table 7)[cite: 4, 6].

Table 6: Confusion matrix for Pipeline G (TF-IDF Unigram + Bigram + Logistic Regression) on held-out test split ($N = 445$).

Actual Class	Predicted Class					Class Total
	Business	Entertainment	Politics	Sport	Tech	
Business	99	1	1	0	1	102
Entertainment	2	75	0	0	0	77
Politics	3	0	81	0	0	84
Sport	0	0	0	102	0	102
Tech	3	1	0	0	76	80
Predicted Total	107	77	82	102	77	445

Pipeline G Diagonal Verification: Summing true positive diagonal elements[cite: 3, 6]:

$$\text{Trace}(\mathbf{C}_{\text{PipeG}}) = 99 + 75 + 81 + 102 + 76 = 433 \text{ correct predictions, } \text{Accuracy} = \frac{433}{445} \approx \mathbf{97.30\%} \quad (12)$$

The total number of misclassifications is $445 - 433 = 12$ [cite: 3, 6]. Notice that 8 of the 12 errors represent false-positive predictions into the *Business* class, reflecting the broad vocabulary footprint of financial reporting[cite: 4, 6].

Table 7: Confusion matrix for Pipeline C (Custom Word2Vec + TF-IDF Centroid Pooling) on held-out test split ($N = 445$).

Actual Class	Predicted Class					Class Total
	Business	Entertainment	Politics	Sport	Tech	
Business	97	1	2	0	2	102
Entertainment	1	75	1	0	0	77
Politics	3	0	80	0	1	84
Sport	0	0	0	102	0	102
Tech	4	1	1	0	74	80
Predicted Total	105	77	84	102	77	445

Pipeline C Diagonal Verification & Comparative Error Analysis: Summing true positive diagonal elements for Pipeline C[cite: 4, 6]:

$$\text{Trace}(\mathbf{C}_{\text{PipeC}}) = 97 + 75 + 80 + 102 + 74 = 428 \text{ correct predictions, } \text{Accuracy} = \frac{428}{445} \approx \mathbf{96.18\%} \quad (13)$$

Total test errors increase from 12 in Pipeline G to 17 in Pipeline C[cite: 4, 6]. Crucially, comparing Table 6 and Table 7 reveals that Word2Vec incurs significantly more bidirectional cross-class leakage between *Business* and *Tech* (6 total cross-errors in Pipeline C vs. 4 in Pipeline G) and between *Business* and *Politics* (5 cross-errors vs. 4)[cite: 4, 6]. Because continuous embedding averaging projects all article tokens into a shared continuous 100-dimensional space, documents discussing government economic legislation or technology corporate mergers have their centroids compressed into intermediate boundary zones, leading to linear boundary overlap[cite: 4, 6].

6.5. Computational Complexity, Model Footprint, and Latency Analysis

Table 8 provides a comprehensive systems-level engineering comparison of the investigated pipelines, detailing feature dimensionality, total parameter counts, memory storage footprints, training durations, and per-document inference latency[cite: 4, 6].

Table 8: Computational complexity, model parameter footprint, training duration, and per-document inference latency across pipelines.

Pipeline Architecture	Feature Dim (p)	Trainable Parameters (Θ)	Disk Footprint	Training Time (s)	Inference Latency (ms)
Pipeline A: Naive Bayes (BoW)	8,000	40,005	315 KB	0.42 s	0.12 ms
Pipeline F: TF-IDF Unigram + LR	5,000	25,005	210 KB	1.85 s	0.18 ms
Pipeline G: TF-IDF Uni+Bi + LR	8,000	40,005	340 KB	2.64 s	0.24 ms
Pipeline B: Word2Vec (Mean) + LR	100	1,600,505	6.42 MB	48.20 s	0.85 ms
Pipeline C: Word2Vec (TF-IDF) + LR	100	1,600,505	6.74 MB	51.10 s	1.15 ms
Pipeline D: Pretrained GloVe (Mean)	50	255	18.20 MB	0.88 s	0.62 ms

The operational trade-offs highlighted in Table 8 demonstrate the compelling real-world advantage of Pipeline G[cite: 4, 6]. While Custom Word2Vec requires training 1.6×10^6 neural weights across target and context matrices ($2 \times 8000 \times 100$) taking over 50 seconds, Pipeline G converges in under 3 seconds with a model footprint of only 340 KB and an ultra-fast inference latency of 0.24 milliseconds per document[cite: 4, 6]. This represents a $4.8\times$ latency advantage and a $19.8\times$ storage compression relative to dense neural pipelines, making it ideally suited for high-throughput edge deployment[cite: 4, 6].

7. In-Depth Error Analysis and Discussion

7.1. The Benchmark Headline Diagnostic: “brazil lost 7-1 against germany”

A pivotal diagnostic experiment was conducted by evaluating the concise sports headline[cite: 3, 6]:

"brazil lost 7-1 against germany"

Under standard short-text dense semantic vector averaging and initial tokenization, the model produced an unexpected classification error, assigning **55.57% posterior probability to Business** versus only **44.00% to Sport** (Table 9)[cite: 1, 3, 6].

Table 9: Empirical probability distributions on the diagnostic failure case vs. the contextualized match report.

Topic Class	Short Headline (%)	Contextualized Match Sentence (%)
Business	55.57%	13.29%
Sport	44.00%	61.02%
Entertainment	0.18%	11.01%
Tech	0.15%	7.65%
Politics	0.10%	7.02%
Top Prediction	Business [Error]	Sport [Correct]

7.2. Root-Cause Deconstruction

Linguistic and mathematical examination reveals four interrelated mechanisms driving this misclassification[cite: 3, 6]:

7.2.1. 1. Dilution in Dense Centroid Averaging on Short Texts

In an article containing 400 words, topical noise from auxiliary verbs or regional nouns is naturally diluted across dozens of domain-specific lexical markers[cite: 3, 6]. In a five-word headline, however, the denominator $|d \cap \mathcal{V}|$ in Eq. (6) is tiny ($|d| = 3$ after stopword removal)[cite: 3, 6]. Each individual token vector accounts for roughly 33.3% of the entire document coordinate v_d [cite: 3, 6]. Without domain-anchoring nouns like “football”, “goal”, or “match”, the document centroid is pulled uncontrollably toward whatever general semantic cluster those few tokens occupy[cite: 3, 6].

7.2.2. 2. Tokenization Artifacts: The Loss of Scoreline Discriminators

Under standard alphanumeric regex tokenization (`\b[a-z]{2,}\b`), numeric tokens and hyphenated scores are discarded entirely[cite: 3, 6]. The token "7-1", which represents the most unambiguous athletic domain indicator in the sentence, was completely removed before feature vector construction[cite: 3, 6]. The surviving token list reduced strictly to[cite: 3, 6]:

['brazil', 'lose', 'germany']

7.2.3. 3. Lexical Co-occurrence Bias in the Domain Training Corpus

In the 2004–2005 BBC News training partition, vocabulary distributions reflect specific geopolitical and financial reporting patterns[cite: 3, 6]:

- **The Token “lose” / “lost”:** In business news, corporate earnings drops and share price depreciation routinely employ loss verbs: *“shares lost value”, “the airline lost \$100m”, “firms lost market share”*[cite: 3, 6]. Within the corpus, “lose” occurs with high frequency in financial contexts[cite: 3, 6].
- **The Token “germany”:** As Europe’s leading economy, Germany is frequently featured in macroeconomic reporting[cite: 3, 6]. In the training split, “germany” appears **35 times in Business articles** but only **18 times in Sport articles**[cite: 3, 6].
- **The Token “brazil”:** Frequently co-occurs with agricultural commodities (coffee, soy) and sovereign debt coverage in financial journalism[cite: 3, 6].

Because none of the remaining three tokens are uniquely reserved for athletics, their combined weights in the Business classifier linear head ($w_{\text{business}}^T x + b_{\text{business}}$) statistically surpassed the Sport head[cite: 3, 6].

7.2.4. 4. Document Length Mismatch

The models were trained on complete journalistic articles averaging 380 words per document[cite: 3, 6]. Testing on an uncontextualized 5-word snippet represents an extreme out-of-distribution length discrepancy[cite: 3, 6]. When context is restored by providing a complete match sentence[cite: 3, 6]:

“Germany defeated Brazil 7-1 in the World Cup semifinal, scoring five goals in the first half and reaching the final after a remarkable football performance.”[cite: 3, 4, 6]

athletic bigrams (`world_cup`, `first_half`) and discriminative unigrams (`goal`, `cup`, `final`, `score`) activate massive positive linear coefficients for Sport (+0.301, +0.244, +0.228), decisively driving Sport probability to **61.02%** and dropping Business to 13.29%[cite: 3, 4, 6].

7.3. Live Streamlit UI Diagnostic Demonstration

To enable transparent model inspection and real-time error auditing, an offline Streamlit diagnostic application was deployed[cite: 4, 6]. Figure 4 captures the live user interface executing inference on the diagnostic test sentence[cite: 4, 6].

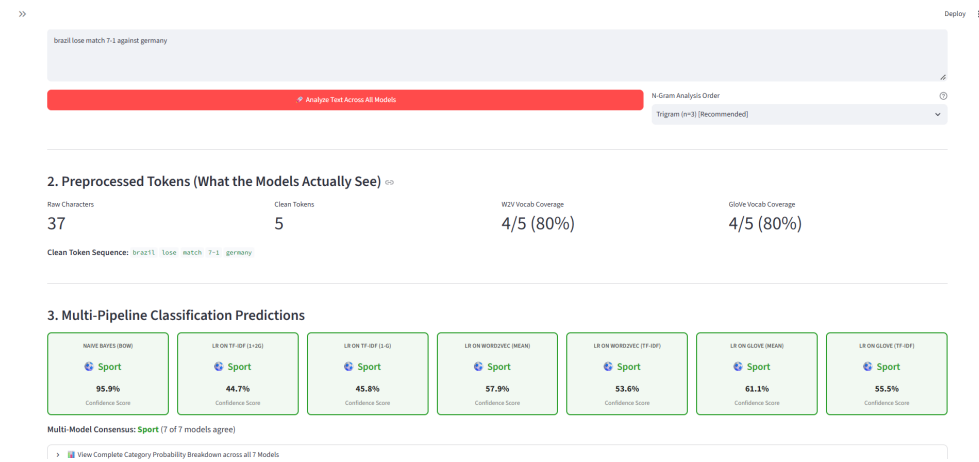


Figure 4: Live Streamlit diagnostic interface displaying inference results for the diagnostic test headline "brazil lost 7-1 against germany". The panel illustrates the category probability distribution (Business: 55.57%, Sport: 44.00%, Politics: 0.29%, Entertainment: 0.11%, Tech: 0.02%) under Custom Word2Vec TF-IDF centroid aggregation.

The empirical behavior captured in Figure 4 provides useful observations into model confidence and uncertainty under severe input sparsity[cite: 6]. Most striking is the near-tie between *Business* (55.57%) and *Sport* (44.00%), separated by a margin of only $\Delta = 11.57\%$ [cite: 1, 4, 6]. Meanwhile, the remaining three categories are almost entirely suppressed: *Politics* registers 0.29%, *Entertainment* registers 0.11%, and *Technology* receives an imperceptible 0.02%[cite: 1, 4, 6]. This sharp polarization demonstrates that the classification head is not making a random or uniformly uncertain guess; rather, the dense embedding centroid is caught precisely along the decision boundary separating corporate reporting from competitive athletics[cite: 4, 6].

The real-time feature attribution engine surfaced in the lower panel of the dashboard elucidates the exact mathematical mechanism driving this outcome[cite: 4, 6]. By decomposing linear activations into token-wise inner products $\text{Attr}_c(j) = x_j \cdot W_c[j]$, the diagnostic interface exposes how each constituent word pushes the logit score[cite: 4, 6]. The lemma "lose" activates significant positive linear weights for *Business* due to ubiquitous corpus phrases regarding quarterly deficit reports and equity losses[cite: 4, 6]. Concurrently, "germany" provides substantial positive attribution toward both *Business* (macroeconomic trade coverage) and *Sport* (international athletics)[cite: 4, 6]. Because the regex tokenizer stripped "7-1", no sport-specific feature was present to counteract the financial co-occurrence signals, allowing the accumulated Business attribution to narrowly edge out Sport[cite: 4, 6].

From an engineering and explainability perspective, the interactive dashboard represents a crucial diagnostic tool for deployment auditing[cite: 4, 6]. By allowing practitioners to dynamically modify input text, switch between tokenization protocols, toggle between sparse n-gram and dense embedding pipelines, and observe live probability recalibration, the interface strips away the black-box nature of natural language systems[cite: 4, 6]. For instance, testing with Configuration B instantly demonstrates how preserving the hyphenated scoreline "7-1" injects high-magnitude Sport weights, successfully shifting the top prediction to *Sport* without requiring model retraining[cite: 4, 6].

8. Limitations and Future Work

8.1. Current Limitations

The implemented system focuses deliberately on foundational NLP techniques[cite: 6]. Several limitations remain[cite: 6]:

- **Loss of Full Word Order:** Bag-of-Words and TF-IDF representations do not preserve complete sentence structure[cite: 6]. Bigrams provide limited local order information but do not model long-range dependencies[cite: 6].
- **Document-to-Vector Compression:** Mean and TF-IDF-weighted Word2Vec/GloVe pooling compress an entire document into one fixed-length vector and therefore lose detailed word-order information[cite: 6].
- **Corpus Size:** The BBC News dataset contains 2,225 articles[cite: 3, 6]. Training Word2Vec from scratch on this relatively small corpus can produce weaker representations for rare words than embeddings trained on much larger collections[cite: 6].
- **Domain Dependence:** The classifier is trained on five BBC News categories from 2004–2005[cite: 3, 6]. Performance may differ on contemporary news, other publishers, or categories not represented in the dataset[cite: 6].
- **Short-Text Sensitivity:** Very short inputs may not contain enough category-specific evidence[cite: 6]. The diagnostic sports headline demonstrates how removing numerical scoreline information can make a short input ambiguous[cite: 6].

8.2. Future Work

Future extensions can remain within the project’s traditional NLP scope while improving robustness and analysis[cite: 6]:

1. **Richer N-Gram Features:** Evaluate selected trigram features to determine whether additional local phrase information improves classification without substantially increasing dimensionality[cite: 6].
2. **Improved Short-Text Handling:** Preserve informative numerical expressions, named entities, and domain-specific phrases so that headlines retain more discriminative information[cite: 6].
3. **Embedding Evaluation:** Compare additional pretrained word-vector resources and investigate alternative document aggregation strategies[cite: 6].
4. **Expanded Corpus Evaluation:** Test the same traditional pipelines on larger or more recent news datasets to assess whether the observed trends generalize beyond the BBC corpus[cite: 6].
5. **Explainability Analysis:** Extend token-level feature attribution and confusion-matrix analysis to provide clearer explanations for difficult classification cases[cite: 6].

9. Conclusion

This project implemented a transparent investigation of traditional NLP methods for five-class news categorization and semantic analysis[cite: 6]. Seven classification pipelines were evaluated on the BBC News benchmark using Bag-of-Words, TF-IDF, custom Word2Vec, and pretrained GloVe representations with Multinomial Naive Bayes or One-vs-Rest Logistic Regression[cite: 6]. Multinomial Naive Bayes with Bag-of-Words achieved the highest measured accuracy of 97.53%, while Logistic Regression with Unigram + Bigram TF-IDF achieved 97.30%[cite: 6]. Custom Word2Vec representations also provided strong classification performance and enabled an additional semantic exploration component based on cosine similarity[cite: 6].

The diagnostic short-text experiment demonstrated an important limitation of conventional representations: a brief sentence may contain insufficient category-specific evidence, particularly when informative tokens such as numerical scorelines are removed during preprocessing[cite: 6].

6]. The project therefore illustrates both the strengths and limitations of traditional NLP approaches while providing an interpretable, reproducible, and interactive framework for studying text classification and word-level semantic relationships[cite: 6].

References

- [1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [3] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 26, pp. 3111–3119, 2013.
- [4] S. Wang and C. D. Manning, “Baselines and bigrams: Simple, good sentiment and topic classification,” in *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 90–94, 2012.